Лекция 5 Тестирование данных, моделей и pipeline

Методические материалы курса · учебный год 2026–2027

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР5 «Тестирование данных, модели и pipeline»

Цель лекции

Показать, что тестирование ML-системы проверяет не только функции Python, но и данные, временные границы, артефакт модели и связи компонентов. После занятия студент должен уметь построить пирамиду тестов, определить проверяемые инварианты, локализовать отказ и объяснить, почему высокая метрика не доказывает инженерную корректность pipeline.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Почему обычных unit-тестов недостаточно	Разобрать корректный код на ошибочных данных
10–25 мин	Пирамида тестов ML-системы	Разнести проверки по unit, contract, model и integration
25–40 мин	Fixtures и параметризация	Построить минимальные положительные и отрицательные данные
40–55 мин	Тестирование времени и утечки	Показать нарушение хронологической границы
55–68 мин	Метрики и артефакт модели	Проверить предусловия и инварианты bundle
68–80 мин	Детерминированность и flaky tests	Разделить воспроизводимость и совпадение метрики
80–90 мин	Defect injection и ЛР5	Продемонстрировать тест, который обязан упасть

1 Что именно мы тестируем

Начните с двух утверждений: «функция выполнилась без исключения» и «система выдаёт корректный воспроизводимый результат». Между ними большой разрыв. В ML-системе код может быть синтаксически правильным, а выборка — содержать будущее, метка — попасть в признаки, validation — состоять из одного класса, bundle — потерять порядок признаков.

Тест — это исполняемая формулировка ожидания. Хороший тест отвечает на три вопроса:

1. Какой риск контролируется?
2. Какой минимальный вход воспроизводит этот риск?
3. Какой наблюдаемый сигнал означает отказ?

Фраза «проверить модель» слишком широка. Формулировка «при validation только одного класса расчёт ROC AUC завершается понятным ValueError» уже задаёт вход, действие и ожидаемый результат.

2 Пирамида тестов ML-системы

Слой	Объект	Пример	Частота
Unit	чистая функция	расчёт метрик и проверка threshold	каждый commit
Data contract	схема и значения	диапазон DepHour, обязательные поля	каждый pull request
Model	сериализованный bundle	features, version, диапазон predict_proba	каждый pull request
Integration	связь компонентов	validate → train → load → predict	каждый pull request
API	внешний контракт	Pydantic, status code, response	перед контейнеризацией
End-to-end	пользовательский путь	контейнер и реальный HTTP-запрос	перед release

Чем выше слой, тем реалистичнее проверка, но тем медленнее диагностика. Один end-to-end не заменяет unit-тесты: он обнаруживает проблему, но часто не объясняет её источник.

3 Инвариант вместо конкретного числа

Инвариант — свойство, которое должно сохраняться для допустимого набора входов. Для нашего проекта:

- вероятность конечна и находится от 0 до 1;
- порядок признаков совпадает с bundle["features"];
- train расположен во времени раньше validation, а validation — раньше test;
- в каждой части присутствуют оба класса;
- версия, параметры и границы split сохраняются рядом с моделью;
- временные файлы теста не меняют эталонные данные.

Не следует без основания фиксировать точную метрику небольшой случайной модели. После безопасного обновления scikit-learn или изменения реализации число может немного измениться, хотя контракт остаётся верным. Используйте pytest.approx там, где допустима численная погрешность, и точное сравнение там, где проверяется схема или версия.

4 Arrange, Act, Assert

Предложите студентам читать тест как короткий эксперимент:

1. Arrange — подготовить минимальный вход и ожидаемое поведение.
2. Act — выполнить одно проверяемое действие.
3. Assert — проверить наблюдаемый результат или исключение.

Если в Arrange запускается половина системы, а Assert содержит только assert result, тест трудно диагностировать. Имя должно объяснять условие и результат: test_metrics_reject_single_class, а не test_metrics_2.

5 Fixtures и изоляция

Fixture предоставляет тесту состояние с понятным временем жизни. tmp_path создаёт отдельную директорию и устраняет зависимость от файлов предыдущего запуска. Общая fixture корректных данных снижает дублирование, но не должна превращаться в неявный глобальный объект, который тесты меняют по очереди.

Покажите два принципа:

- fixture возвращает новый или безопасно копируемый объект;

- тест явно создаёт нарушение, которое проверяет.

Слишком большая fixture скрывает причину отказа. Для проверки диапазона часа достаточно нескольких строк, а не полного учебного набора.

6 Параметризация отрицательных сценариев

Параметризация позволяет применить одно правило к нескольким независимым входам. Например, проверка должна отвергать DepHour=-1, DepHour=24, неизвестную колонку и неверный формат даты. Каждому case полезно дать читаемый id, чтобы отчёт был понятен без открытия кода.

Не объединяйте разные причины отказа только ради короткого файла. Если сценарии требуют разных действий или разных сообщений, отдельные тесты понятнее.

7 Тестирование времени

Для временных данных случайное перемешивание может создать скрытую утечку. Контракт проверяет:

- строки упорядочены по FlightDate;
- максимальная дата train меньше или равна минимальной дате validation;
- максимальная дата validation меньше или равна минимальной дате test;
- признаки доступны за 60 минут до планового отправления;
- фактическая задержка используется только как отложенная цель.

Попросите аудиторию придумать тест для признака actual_arrival_time. Правильный ответ включает не только имя колонки, но и проверку момента доступности.

8 Тесты функции оценки

До расчёта метрик проверяются предусловия:

- длины меток и вероятностей совпадают и не равны нулю;
- метки бинарны;
- присутствуют оба класса;
- вероятности конечны и находятся в диапазоне от 0 до 1;
- threshold находится строго между 0 и 1.

После этого проверяются ключи результата и диапазоны. Тест идеального ранжирования полезен как простой эталон, но одного положительного сценария недостаточно.

9 Тесты model bundle

Сериализованный файл — это контракт между обучением и сервисом. Bundle должен включать:

Поле	Риск при отсутствии
model	предсказание невозможно
features	сервис меняет порядок входов
version	ответ API нельзя связать с артефактом
parameters	обучение не воспроизводится
split	временные границы теряются
validation_metrics	решение нельзя проверить

Помимо структуры нужен smoke test: загрузить bundle и вызвать predict_proba на строке с признаками в сохранённом порядке.

10 Детерминированность

Одинаковые данные, параметры, код и random seed должны давать согласованный результат. Но тест «после смены seed метрика обязана измениться» неверен: на маленьком наборе она может совпасть. Проверяйте, что seed зафиксирован, два запуска с одним seed совпадают по предсказаниям, а metadata отражает настройку.

Детерминированность не гарантирует корректность. Стабильно воспроизводимая утечка остаётся ошибкой.

11 Flaky tests

Flaky test иногда проходит, иногда падает без изменения проверяемого кода. Типичные причины:

- незафиксированный seed;
- зависимость от порядка тестов;
- общие файлы и порты;
- внешняя сеть;
- ожидание процесса фиксированным sleep;
- слишком строгая проверка float.

Лечение — не автоматический повтор до зелёного статуса. Сначала найдите источник недетерминизма, изолируйте состояние и используйте ожидание наблюдаемого события с timeout.

12 Integration smoke test

Минимальный сквозной тест ЛР5 выполняет validate → train → load → predict. Он не проверяет каждый алгоритмический случай, а подтверждает совместимость компонентов. Артефакт создаётся в tmp_path, загружается тем же способом, что и сервис, а вероятность проверяется на форму и диапазон.

На следующей неделе этот путь будет вынесен за границу процесса: Docker image должен пройти HTTP smoke test. Таким образом, тестовый контракт развивается вместе с системой.

13 Defect injection

Чтобы доказать чувствительность теста, студент временно вносит контролируемый дефект. Например, разрешает DepHour=24 или удаляет поле версии bundle. До изменения студент называет test node ID и ожидаемую причину отказа; после демонстрации возвращает корректный код.

Если тест остаётся зелёным, это полезный результат: обнаружен пробел покрытия. Нельзя оставлять дефект в принимаемой ветке или подменять демонстрацию заранее подготовленным скриншотом.

14 Диагностика отказа

Учите читать traceback сверху вниз по пользовательскому коду, но искать первое полезное место, а не последнюю строку вообще. Диагностическое сообщение содержит:

- имя теста и case;
- ожидаемое и фактическое значение;
- минимальный контекст;
- причину на языке контролируемого риска.

Большой дамп DataFrame редко помогает. Полезнее показать нарушенное поле, строку и правило контракта.

15 Что не является доказательством качества

- процент coverage без связи с рисками;
- тест, который не содержит содержательного assertion;
- проверка только успешного пути;
- один общий скриншот команды;
- JUnit XML, созданный не из принимаемого commit;
- зелёный тест после удаления проверяемой логики;

- мокирование самого объекта, который нужно проверить.

Coverage показывает выполненные строки, но не полноту требований. Используйте его как навигационный сигнал, а не как единственный quality gate.

16 Индивидуальная ответственность

Каждый участник должен владеть одним наблюдаемым риском: создать тест или fixture, провести содержательное review, локализовать отказ либо улучшить диагностическое сообщение. На защите студент запускает собственный тест по node ID и выполняет назначенное изменение входа.

Для команды из пяти человек подходят зоны data contract, evaluation, model bundle, integration и quality review. Для пары роли совмещаются, но артефакт и демонстрация остаются индивидуальными.

17 Вопросы аудитории

1. Почему assert result is not None недостаточно для вероятности?
2. Когда нужен pytest.raises?
3. Почему общий временный файл делает тесты зависимыми?
4. Чем model test отличается от integration test?
5. Что доказывает defect injection?
6. Почему смена seed не обязана менять метрику?
7. Как проверить отсутствие временной утечки?
8. Почему зелёный end-to-end не локализует отказ?

18 Завершение

Сформулируйте итог: надёжность ML-pipeline строится не максимальным числом тестов, а явной связью «риск → инвариант → минимальный вход → диагностический сигнал». ЛР5 фиксирует эту связь в pytest и отчётах, а ЛР6 переносит те же гарантии в контейнерную границу.

Чек-лист преподавателя

- Показаны уровни unit, contract, model, integration и end-to-end.
- Разобраны fixtures, tmp_path, parameterization и pytest.raises.
- Временная утечка сформулирована как тестируемый риск.
- Метрики проверяют предусловия, а не только диапазон ответа.
- Bundle рассматривается как контракт обучения и сервиса.
- Flaky test не «лечится» бесконечным повтором.
- Defect injection выполняется индивидуально и откатывается.
- Лекция связана с переходом к контейнерному smoke test.

Официальные источники

- pytest documentation: https://docs.pytest.org/en/stable/
- pytest fixtures: https://docs.pytest.org/en/stable/how-to/fixtures.html
- pytest parametrization: https://docs.pytest.org/en/stable/how-to/parametrize.html
- scikit-learn metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html